EEBus Initiative e.V.
Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

EEBus Test Specification

SHIP

Version 1.0.0

Cologne, 2026-07-16

Terms of use for publications of EEBus Initiative e.V.

General information
The specifications, particulars, documents, publications and other information provided by the EEBus Initiative e.V. are solely for general informational purposes. Particularly specifications that have not been submitted to national or international standardisation organisations by EEBus Initiative e.V. (such as DKE/DIN-VDE or IEC/CENELEC/ETSI) are versions that have not yet undergone complete testing and can therefore only be considered as preliminary information. Even versions that have already been published via standardisation organisations can contain errors and will undergo further improvements and updates in future.

Liability
EEBus Initiative e.V. does not assume liability or provide a guarantee for the accuracy, completeness or up-to-date status of any specifications, data, documents, publications or other information provided and particularly for the functionality of any developments based on the above.

Copyright, rights of use and exploitation
The specifications provided are protected by copyright. Parts of the specifications have been submitted to national or international standardisation organisations by EEBus Initiative e.V., such as DKE/DIN-VDE or IEC/CENELEC/ETSI, etc. Furthermore, all rights to use and/or exploit the specifications belong to the EEBus Initiative e.V., Deutz-Mülheimer-Straße 183, 51063 Cologne, Germany and can be used in accordance with the following regulations:

The use of the specifications for informational purposes is allowed. It is therefore permitted to use information evident from the contents of the specifications. In particular, the user is permitted to offer products, developments and/or services based on the specifications.

Any respective use relating to standardisation measures by the user or third parties is prohibited. In fact, the specifications may only be used by EEBus Initiative e.V. for standardisation purposes. The same applies to their use within the framework of alliances and/or cooperations that pursue the aim of determining uniform standards.

Any use not in accordance with the purpose intended by EEBus Initiative e.V. is also prohibited.

Furthermore, it is prohibited to edit, change or falsify the content of the specifications. The dissemination of the specifications in a changed, revised or falsified form is also prohibited. The same applies to the publication of extracts if they distort the literal meaning of the specifications as a whole.

It is prohibited to pass on the specifications to third parties without reference to these rights of use and exploitation.

It is also prohibited to pass on the specifications to third parties without informing them of the authorship or source.

Without the prior consent of EEBus Initiative e.V., all forms of use and exploitation not explicitly stated above are prohibited.

Table of contents

List of tables

1 Introduction

This document defines a set of tests to verify the conformity of implementations with the EEBus Smart Home IP (SHIP) specification ([SHIP]).

The test cases are designed to validate the normative requirements (marked with keyword SHALL) and recommended behaviors (SHOULD) of the underlying secure communication and connection management layer to guarantee cross-manufacturer interoperability. This includes both positive scenarios and negative tests to ensure robustness and correct error handling.

The primary objective is to verify that a Device Under Test (DUT) correctly implements the foundational SHIP mechanisms. The test cases in this document cover the following core areas:

- mDNS & DNS-SD Service Discovery: Correct handling of SHIP TXT records, service announcements, and mandatory key-value pairs (e.g., SHIP ID, SKI, path).
- Connection Mode Initialization (CMI): Strict processing of the CMI state machine and its fallback mechanisms to prevent protocol incompatibilities.
- SHIP Message Exchange (SME): Validation of connection states including "Hello", "Protocol Handshake", and "PIN Verification", along with rigorous testing of timers and prolongation requests.
- Key Material & Trust Management: Proper handling of SKI evaluation, role uniqueness, and secure connection termination.
- Robustness & Application Data Exchange: Enforcement of JSON parser stability (whitespace handling) and the concurrent processing of the different SME message types (e.g. data exchange vs. connection-management messages).

Successful execution of these tests ensures that the DUT provides a reliable, mutually authenticated, and interoperable transport foundation for any subsequent EEBUS application-layer Use Cases.

1.1 References

1.1.1 EEBUS documents

The following EEBUS specifications and technical documents form the basis for this test specification.

[SHIP]	*Minimum*:	EEBus_SHIP_TS_Specification_v1.0.1.pdf
	Recommended:	EEBus_SHIP_TS_Specification_v1.1.0.pdf
[SHIP_IG]	*Minimum*:	EEBus_SHIP_IG_TransportAndConnectivitiy_V1.0.0.pdf
[SRIP]	*Minimum*:	EEBUS_TS_ShipRequirementsForInstallationProcess_V1.1.0.pdf

1.1.2 Normative references

The normative references listed below contain provisions which, through reference in this text, constitute provisions of this specification.

[RFC 2119]	IETF RFC 2119: Key words for use in RFCs to indicate requirement levels, 1997

[RFC 5246] IETF RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2

[RFC 6455] IETF RFC 6455: The WebSocket Protocol

[RFC 6762] IETF RFC 6762: Multicast DNS

[RFC 6763] IETF RFC 6763: DNS-Based Service Discovery

1.2 Terms, definitions and abbreviations

In this document, the following terms, definitions and abbreviations apply:

CF
Abbreviation for "Configuration". Identifier for a specific pre-condition state required before test execution.

Client
Regarding TCP-based data exchange between two SHIP nodes, the term client fundamentally refers to the TCP connection role. If the term is also used regarding service discovery, "client" stands for a SHIP node that evaluates service announcements of other SHIP nodes (to potentially establish a connection).

Server
Regarding TCP-based data exchange between two SHIP nodes, the term server fundamentally refers to the TCP connection role. If the term is also used regarding service discovery, "server" stands for the SHIP instance that announces a service (because this device must provide a server port).

CMI
Abbreviation for "Connection Mode Initialization". The first protocol phase after connection establishment to ensure format agreement before connection data preparation.

DEV-A
A normative symbol used specifically during the connection termination phase. It represents the device that initiates the connection termination process, regardless of whether it was the TCP server or the TCP client.

DEV-B
A normative symbol used specifically during the connection termination phase. It represents the communication partner of DEV-A (the receiver of the termination request).

DUT
Abbreviation for "Device Under Test". The specific implementation being verified for SHIP compliance.

mDNS / DNS-SD
Multicast DNS / DNS-Based Service Discovery. Mechanisms used to find available SHIP nodes and their services in the local network.

PIN
Personal Identification Number. An optional secret used for SHIP specific verification procedures to improve the trust level.

SHIP
Abbreviation for "Smart Home IP". An IP-based protocol for secure machine-to-machine communication in home automation and further domains.

SHIP Node
A logical device communicating via the described SHIP protocol.

SKI
Abbreviation for "Subject Key Identifier". An identifier derived from a specific public key, used as a cryptographically backed identification and authentication criterion.

SME
Abbreviation for "SHIP Message Exchange". The interface and protocol layer handling the connection states, handshakes, and SHIP message formatting.

Tester
A human operator or an automated external mechanism responsible for performing physical, administrative, or out-of-band interactions on the DUT (e.g., removing a trusted device via the user interface, triggering a reboot, or confirming a pairing code).

Test Tool
The execution engine or test framework instance that simulates the communication partner of the DUT. It handles the SHIP protocol communication and verifies whether the DUT complies with the SHIP rules.

TC
Abbreviation for "Test Case".

Trust Level
A metric expressing the trust in a communication partner, using generic values grouped in categories such as User trust, PKI trust, and Second factor trust.

2 Test suite convention

This chapter defines the technical conventions and formal rules required for the consistent implementation of the SHIP test suite. It establishes the naming schemes, system configurations, and data variables necessary for both manual and automated test execution.

2.1 Test suite identifiers

To ensure consistency and machine-readability, this test suite uses hierarchical naming conventions for all components. The identifiers are self-descriptive:

- Test Case (TC): Named as TC_SHIP_<context>_<number> (Example: TC_SHIP_CMI_001)
- Precondition (PRE): Named as PRE_SHIP_<state> to define stable starting configurations and environment settings. (Example: PRE_SHIP_ConnectionEstablished)

2.2 Preconditions

This section describes the required initial states for test cases. A precondition (PRE) defines a functional milestone that SHALL be reached before the actual test steps can be executed.

PRE_SHIP_Ready_For_Handshake

The Test tool SHIP stack is initialized and ready to initiate or accept a TCP/TLS handshake, but no active session exists yet.

PRE_SHIP_TLS_Established

The TCP connection has been opened, and the TLS 1.2 handshake is successfully completed, including mutual certificate verification. The connection has NOT yet entered the Connection Mode Initialization (CMI) phase. A mutual pre-trust between the test tool and the DUT MUST already exist to reach this condition.

PRE_SHIP_ConnectionEstablished

The SHIP connection has reached the state "connection data exchange", i.e. TCP/TLS handshake, CMI, "Hello", SME "protocol handshake" and PIN verification are completed. A mutual pre-trust between the test tool and DUT must already exist to reach this condition.

PRE_SHIP_Manual_Message_Handling

A special configuration mode for the test tool. Automatic SME protocol replies (like acknowledging a prolongation request or immediately answering a CMI message) are disabled to test timeout

behaviors. This configuration applies exclusively to the SME layer, autonomous replies and timeouts on the underlying TCP or WebSocket layers remain fully active and functional.

PRE_SHIP_PIN_Disabled

The DUT is explicitly configured without a PIN or does not require PIN authentication by design. Its internal configuration requires it to announce pinState = "none" during the SHIP PIN verification phase.

PRE_SHIP_ServerPort

The choice in PAR_shipSvc must be "yes". The DUT MUST have a SHIP server port and announce its SHIP service via mDNS.

PRE_SHIP_TestTool_ClientOnly

The test tool MUST be configured so that it does NOT have a functional SHIP server port (or only a dummy server port that does not accept TCP requests). This ensures the DUT cannot initiate a connection faster than the test tool. The test tool acts as the TCP client.

PRE_SHIP_TestTool_ServerOnly

The test tool MUST be configured so that it does NOT actively initiate a connection to the DUT. It MUST announce its SHIP service via mDNS beforehand and wait passively for the DUT to initiate the TCP connection.

PRE_SHIP_TestTool_ServerAndClient

The test tool MUST be configured to operate simultaneously as SHIP server and SHIP client. It MUST provide a functional SHIP server port and announce its SHIP service via mDNS, so that the DUT can discover it and initiate a connection to the test tool. In addition, the test tool MUST be able to actively initiate a client connection to the DUT.

PRE_SHIP_Mutual_Trust

Mutual trust MUST already be established between the DUT and the test tool before the test execution begins (e.g., by pre-configuring the DUT to trust the test tool's certificate or SKI).

PRE_SHIP_TestTool_Smaller_SKI

The test tool MUST have a self-signed certificate/key pair with an SKI that is strictly SMALLER than the DUT's SKI. This might require a repeated generation of key pairs until a public key with this property is found.

PRE_SHIP_TestTool_Multiple_Instances

The test tool MUST be configured to simulate at least two distinct, independent SHIP instances (e.g., Instance A and Instance B) simultaneously on the network. Each instance MUST possess its own unique cryptographic identity (certificate/SKI) and its own unique SHIP ID, ensuring the DUT treats them as two completely separate communication partners.

PRE_SHIP_Hello_Completed

The test tool autonomously executes the TCP/TLS handshake, the CMI phase, and the SME "hello" phase with the DUT in full compliance with the SHIP specification. Mutual trust is assumed and established (both sides reach phase = "ready"). This precondition ends exactly when both devices have reached the state "HELLO_OK" and are ready to enter the "protocol handshake" phase. From this point onward, automatic protocol replies by the test tool are disabled to allow manual test execution.

PRE_SHIP_Protocol_Handshake_Completed

The test tool and the DUT have successfully established a TCP/TLS connection, completed the CMI and SME "hello" phases, and successfully executed the SME "protocol handshake" phase (agreeing on a format like JSON-UTF8). This precondition ends exactly at the transition point into the "PIN verification" state.

PRE_SHIP_No_Prior_Pairing

No prior pairing or trust relationship exists between the DUT and the test tool at the start of the test.

PRE_SHIP_TestTool_SpoofedCertificate

The test tool is configured with a self-signed certificate whose SKI field does NOT match SHA-1 of the certificate's public key. The test tool announces this (spoofed) SKI value in its mDNS TXT record.

2.3 Test case description

To ensure a uniform evaluation of the SHIP protocol requirements, all Test Cases (TCs) are presented in a standardized tabular format. This structure ensures that test engineers and developers can clearly distinguish between the protocol state, the technical stimulus, and the required system behavior.

The header of each test case provides the administrative and logical context required for execution:

Test case ID
The unique identifier of the test case according to the naming convention.

Description
A concise summary of the test objective and the specific protocol mechanism being verified.

Roles
Defines the functional SHIP protocol roles assumed by the DUT and the Test tool. Valid values are "Server", "Client", "Server & client", or "Server or client", based on the TCP connection role. "Server & client" denotes that the DUT assumes both roles simultaneously within the test case (one connection per role). "Server or client" denotes that the test case is executed once, with the DUT in either role; the test tool assumes the complementary role.

Applicability
Defines which devices the test applies to (e.g., "Applicable to any DUT").

Referenced Requirements
References the centralized requirement identifiers (e.g., [SHIP-TS-CMI-01]).

Definitions
An optional block that introduces symbols used within a single test case and their meaning; typically cryptographic artefacts such as certificates, public/private keys, or calculated values (e.g., CERT_T1, PUB_T1, SKI_T1 = SHA-1(PUB_T1)). It is provided only for test cases that rely on such symbols, in order to keep the preconditions and test steps concise and unambiguous. These definitions have local scope: they apply only within the test case in which they appear and are distinct from the global terms in section 1.2.

Precondition
The required initial system state that SHALL be successfully established before the first test step can be executed.

Entry point
Used only for test cases with a timing dependency between the completion of the precondition and the first test step: once the state established by the referenced precondition is reached, the first test step MUST be executed immediately (e.g. because a timer in the DUT is already running from that point). The field names the precondition that defines this state. Test cases without such a timing dependency do not use this field.

Test Step [X]
Sequentially numbered execution steps. Each step is divided into Action/Execution (the technical stimulus) and Expected Result (the observable protocol-level reaction).

2.4 Standardized execution notation

Since this specification defines behavior on a protocol level, the test steps avoid tool-specific automation keywords. Instead, a standardized execution notation is used to describe the interactions between the test tool and the DUT.

Execution verbs:

- "sends": The test tool actively constructs and transmits a SHIP message over the active TCP/TLS connection.
- "waits for": The test tool passively listens on the active TCP/TLS connection for an incoming SHIP message matching the specified criteria. The duration of this listening phase is bounded by the applicable timeout rules (default or explicit) defined in this section.
- "silently ignores": The DUT accepts the received message without triggering an error, does not drop the TCP/TLS connection, and proceeds with the normal control flow without sending a visible SHIP protocol response.

Message definitions

When a test step specifies an action like "sends an SME 'hello' message", it refers to the MessageType (e.g., init, control, data, end) and the corresponding SME (SHIP Message Exchange) structure defined in [SHIP] section 13.4.

WebSocket framing

SHIP messages are exchanged exclusively in WebSocket binary data frames (opcode 0x2). WebSocket control frames (ping, pong, close; opcodes 0x8–0xA) operate at the WebSocket layer and are not SHIP messages. They are therefore never counted as a SHIP message in an Expected Result, and the test tool MAY use ping/pong to determine connection liveness.

Timeout determination

1. If a test step's Expected Result defines an explicit timeout (e.g. "up to 22s", "CmiTimeout"): The test tool marks the test case as failed if the DUT exceeds this duration.
2. By default, the test tool applies timeout rules as specified by the SHIP specification. As long as a timeout value from the specification is not superseded by an excplicit timeout from a test step: The test tool marks the test case as failed if the DUT exceeds this duration in case of mandatory events or behaviours.

2.5 Parameters for the test execution

This section specifies the configuration sets for the test keywords. To ensure a clear distinction between protocol-specific data and test automation instructions, the parameters are divided into the structural components of a SME message:

- MessageType: the 1-byte SHIP message type (e.g., 0x00 = init).
- MessageValue: the payload (or at least parts of the payload) of the SHIP message (e.g., CmiHead, connectionHello.phase).

The following parameters define the standardized message payloads used during test execution:

PAR_cmiValidInit

- MessageType = 0x00 (init)
- MessageValue: CmiHead = 0x00

PAR_msgEmptyMessageValue

- MessageType = 0x02 (data)
- MessageValue: <EMPTY> (The MessageValue is completely omitted, resulting in a 0-byte payload)

PAR_msgUnknownMessageType

- MessageType = 0xFF (Unknown/RFU message type)
- MessageValue: MessageValue = 0x00 (A valid 1-byte payload to ensure the DUT does not reject the message due to an empty MessageValue)

PAR_cmiInvalidMessageType

- MessageType = 0x01 (control)
- MessageValue: CmiHead = 0x00

PAR_cmiInvalidCmiHead

- MessageType = 0x00 (init)
- MessageValue: CmiHead = 0x01 (Invalid CmiHead, greater than 0)

PAR_helloStateReady

- MessageType = 0x01 (control)
- MessageValue: connectionHello.phase = "ready"

PAR_helloProlongationRequest
- MessageType = 0x01 (control)
- MessageValue: connectionHello.phase = "pending"
- MessageValue: connectionHello.prolongationRequest = true

PAR_helloStatePending
- MessageType = 0x01 (control)
- MessageValue: connectionHello.phase = "pending"
- MessageValue: connectionHello.waiting = 120000 (ms)

PAR_protAnnounceMax
- MessageType = 0x01 (control)
- MessageValue: messageProtocolHandshake.handshakeType = "announceMax"
- MessageValue: messageProtocolHandshake.version.major = <major part of ${PAR_testToolShipVersion}>
- MessageValue: messageProtocolHandshake.version.minor = <minor part of ${PAR_testToolShipVersion}>
- MessageValue: messageProtocolHandshake.formats.format = "JSON-UTF8"

PAR_pinStateNone
- MessageType = 0x01 (control)
- MessageValue: connectionPinState.pinState = "none"

Test setup parameters

The following parameter MUST be defined by the manufacturer prior to test execution:

PAR_shipSvc (Mandatory)

Select if the DUT announces its SHIP service via mDNS. Possible choices are "yes" (recommended) or "no". The choice "no" is only permissive if the DUT conforms to the German "FNN Lastenheft Steuerbox" that permits the "Steuerbox" (control box) not to announce its SHIP service. In this case, the vendor has to specify this conformance.

PAR_helloProlongationCount (Mandatory)

Defines the exact number of prolongation requests the test tool SHALL execute during the connection mode "hello" test (in TC_SHIP_HELLO_002). The manufacturer MUST specify a value of at least 2. If a high value is specified, the test duration will increase accordingly.

PAR_testToolShipVersion (Mandatory, test tool configuration)

Defines the maximum SHIP specification version the test tool is configured to announce during the protocol handshake. Default: "1.0". It MUST share the same major version as the DUT (PAR_shipVersion). SHIP versions with the same major number are compatible (e.g. a 1.0 DUT interoperates with a 1.1 test tool). The negotiated version is the greatest version supported by both.

PAR_shipVersion (Mandatory)

Defines the maximum SHIP specification version supported by the DUT (e.g., "1.0" or "1.1"). This value is used to verify the version negotiation during the protocol handshake.

PAR_initiateClose (Mandatory)

Select if the DUT announces a connection termination with an SME "close" message (phase = "announce", including a valid "reason" and "maxTime") before closing the TCP/TLS connection. Possible choices are "yes" (recommended) or "no". A DUT that closes the TCP/TLS connection directly, without a prior "close" announcement, selects "no" and is therefore not applicable to test cases requiring PAR_initiateClose set to "yes".

PAR_shipId (Mandatory if PAR_shipSvc = "no")

The manufacturer declares the DUT's unique SHIP ID. Used to verify the accessMethods.id field returned by the DUT.

PAR_queryAccessMethods (Mandatory)

Select if the DUT sends an SME "Access methods request" message when it reaches the SME state "connection data exchange". Possible choices are "yes" or "no".

PAR_clientDetailedDiscoverySupported (Mandatory)

Select if the DUT actively initiates a SPINE nodeManagementDetailedDiscoveryData read request after establishing the connection (i.e. the DUT acts as a Detailed Discovery client). Possible choices are "yes" or "no".

2.6 Acceptance criteria

This section defines the general acceptance criteria used by the test tool to determine whether a test case passes or fails. A test case is only considered passed if all relevant criteria outlined in the table below are successfully met.

Category	Criteria for pass	Criteria for fail
1. Message verification	The DUT sends the expected SME message and all VERIFY conditions from the active parameter group are met.	The DUT sends an unexpected MessageType, or the payload violates the VERIFY conditions.
2. Timeout compliance	The DUT executes the expected action and sends the corresponding message within the applicable timeout as stated in the test step (see section 2.4).	The applicable timeout (see section 2.4) expires without the expected reaction from the DUT.
3. Connection stability	The underlying TCP/TLS connection remains stable and active during the entire test case execution.	The DUT unexpectedly closes the TCP/TLS connection or stops responding to protocol-level handshakes.
4. Message sequence	The DUT adheres to the required protocol state machines (e.g. CMI, SME Hello, protocol handshake).	The DUT violates sequence rules (e.g. skipping the CMI evaluation phase).

Table 1: General acceptance criteria for test execution

3 Requirement overview and mapping to test cases

This chapter provides an overview of the normative requirements defined in the EEBus Smart Home IP (SHIP) Specification ([SHIP]) and their mapping to the test cases defined in this document. This bidirectional traceability ensures comprehensive test coverage and facilitates impact analysis by directly linking each protocol requirement to its corresponding verification steps.

3.1 Requirement to test case mapping

This section establishes the identification system for the foundational protocol requirements. To ensure seamless traceability, all requirements are mapped to their normative sources.

The requirement texts in this table are short summaries of the referenced source sections and are not necessarily verbatim quotes.

Req-ID	Source	Requirement (excerpt)	Test case ID
SHIP-TS-MDNS-01	SHIP 7.3.2 SRIP 2.2	DUT's mDNS TXT record SHALL contain all mandatory key-value pairs and adhere to UTF-8 formatting.	TC_SHIP_MDNS_001 (4.1.1)
SHIP-TS-CONN-01	SHIP 12.2.2	The SHIP node with the larger SKI value SHALL keep the most recent connection open and actively close all other connections.	TC_SHIP_CONN_001 (4.2.1)
SHIP-TS-ROLE-01	SHIP 13.4.1	DUT is permitted to have different roles (client/server) per connection (Role Polymorphism).	TC_SHIP_ROLE_001 (4.3.1) TC_SHIP_ROLE_002 (4.3.2) TC_SHIP_ROLE_003 (4.3.3)
SHIP-TS-SEC-01	SHIP 12.2	A SHIP node SHALL compute SHA-1 of the public key extracted from a received certificate and compare it against the trusted SKI of the peer. If they do not match, the node SHALL reject the certificate and abort the TLS handshake.	TC_SHIP_SEC_001 (4.4.1)
SHIP-TS-SEC-02	SHIP 12.2	On reconnection, a SHIP node SHALL re-verify that SHA-1 of the presented public key matches the trusted SKI, and SHALL reject the certificate (abort the TLS handshake) if it does not match.	TC_SHIP_SEC_002 (4.4.2)
SHIP-TS-MSG-01	SHIP 13.4.2	DUT SHALL require a MessageValue (minimum 1 octet) for a valid SHIP message.	TC_SHIP_MSG_001 (4.5.1)
SHIP-TS-MSG-02	SHIP 13.4.2	DUT SHALL silently discard or close the channel if an unknown MessageType is received.	TC_SHIP_MSG_002 (4.5.2)

Req-ID	Source	Requirement (excerpt)	Test case ID
SHIP-TS-CMI-01	SHIP 13.4.3	DUT entering CMI EVALUATE SHALL correctly process MessageType = 0 and CmiHead = 0.	TC_SHIP_ROLE_001 (4.3.1) TC_SHIP_ROLE_002 (4.3.2)
SHIP-TS-CMI-02	SHIP 13.4.3	DUT SHALL close the connection if an invalid CMI message (MessageType ≠ 0) is received.	TC_SHIP_CMI_001 (4.6.1) TC_SHIP_CMI_002 (4.6.2)
SHIP-TS-CMI-03	SHIP 13.4.3	DUT SHALL close the connection if a CmiHead value greater than 0 is received.	TC_SHIP_CMI_005 (4.6.5) TC_SHIP_CMI_006 (4.6.6)
SHIP-TS-CMI-04	SHIP 13.4.3	DUT SHALL close the connection if no CMI message is received before the CmiTimeout expires.	TC_SHIP_CMI_003 (4.6.3) TC_SHIP_CMI_004 (4.6.4)
SHIP-TS-HELLO-01	SHIP 13.4.4.1.3	DUT SHALL set a Wait-For-Ready-Timer when entering SME connection state "Hello".	TC_SHIP_HELLO_001 (4.7.1)
SHIP-TS-HELLO-02	SHIP 13.4.4.1.3	DUT SHALL accept incoming prolongation requests and increase its Wait-For-Ready-Timer.	TC_SHIP_HELLO_002 (4.7.2)
SHIP-TS-HELLO-03	SHIP 13.4.4.1.3	DUT SHALL execute the common "abort" procedure if its Wait-For-Ready-Timer expires.	TC_SHIP_HELLO_003 (4.7.3)
SHIP-TS-HELLO-04	SHIP 13.4.4.1.3	DUT in state "READY" receiving a "pending" message without "prolongationRequest" SHALL silently ignore the message.	TC_SHIP_HELLO_004 (4.7.4)
SHIP-TS-PROT-01	SHIP 13.4.4.2.2	DUT MUST support the format JSON-UTF8 for the protocol handshake.	TC_SHIP_PROT_001 (4.8.1) TC_SHIP_PROT_002 (4.8.2)
SHIP-TS-PROT-02	SHIP 13.4.4.2.3	DUT SHALL execute "abort" procedure with error type "timeout" if the Wait-Timer expires.	TC_SHIP_PROT_003 (4.8.3) TC_SHIP_PROT_004 (4.8.4)
SHIP-TS-PROT-03	SHIP 13.4.4.2.3	DUT SHALL execute the common 'abort' procedure with error type 'unexpected message' if it receives a valid but unexpected SME message.	TC_SHIP_PROT_005 (4.8.5) TC_SHIP_PROT_006 (4.8.6)
SHIP-TS-PIN-01	SHIP 13.4.4.3.5.1	DUT SHALL set its pinState to 'none' and proceed to the 'Connection data exchange' state if it does not require a PIN.	TC_SHIP_PIN_001 (4.9.1)
SHIP-TS-TERM-01	SHIP 13.4.8.1.2	DUT initiating a termination SHALL close the connection latest after its announced "maxTime".	TC_SHIP_TERM_001 (4.10.1)

Req-ID	Source	Requirement (excerpt)	Test case ID
SRIP-TS-TXT-01	SRIP 2.2	DUT's SHIP TXT record SHALL contain all mandatory key-value pairs (e.g., brand, model, cat) and adhere to UTF-8 formatting.	TC_SHIP_MDNS_001 (4.1.1)
SHIP-IG-01	SHIP-IG 2.2	A SHIP node's JSON parser SHALL correctly process structural whitespace characters (Space, Horizontal tab, LF, CR) as permitted by the JSON specification and SHALL NOT expect minified payloads.	TC_SHIP_MSG_003 (4.5.3)
SHIP-IG-02	SHIP-IG 2.1	A SHIP node SHALL process application-layer SPINE messages concurrently with SME-layer access methods requests. Incoming SPINE messages MUST be passed to the application layer immediately, even if an SME access methods query is pending.	TC_SHIP_AMDATA_001 (4.11.2) TC_SHIP_AMDATA_002 (4.11.3)
SHIP-IG-03	SHIP-IG 2.1	A SHIP node SHALL NOT postpone the processing of an incoming SME "access methods" message while awaiting the response to a pending SME "data" (SPINE) message.	TC_SHIP_AMDATA_003 (4.11.4)
SHIP-TS-DATA-01	SHIP 13.4.6	The DUT SHALL set the accessMethods.id field of its SME "access methods" message to its own unique SHIP ID.	TC_SHIP_AM_001 (4.11.1)
SHIP-TS-ACC-01	SHIP 13.4.6	The recipient of an SME 'Access methods request' message SHALL respond with an SME 'Access methods' message.	TC_SHIP_MSG_003 (4.5.3)

Table 2: Requirement to test case mapping

3.2 Test case to requirement mapping

This section provides a reverse mapping from the test cases defined in this document to the normative requirements. It allows the identification of the specific rules verified by a given test case depending on the role of the DUT.

Status definition (M/O)

This column indicates the implementation requirement for the specified DUT role:

- M (Mandatory): The requirement SHALL be implemented. The corresponding test case must be passed to achieve compliance.

509 - O (Optional): The requirement MAY be implemented. If the manufacturer chooses to support
510 this feature, it must comply with the specification, and the corresponding test case must be
511 passed.

Test case ID	DUT	M/O	Req-ID
TC_SHIP_MDNS_001	Server	M	SHIP-TS-MDNS-01 SRIP-TS-TXT-01
TC_SHIP_CONN_001	Server & client	M	SHIP-TS-CONN-01
TC_SHIP_ROLE_001	Server	M	SHIP-TS-ROLE-01 SHIP-TS-CMI-01
TC_SHIP_ROLE_002	Client	M	SHIP-TS-ROLE-01 SHIP-TS-CMI-01
TC_SHIP_ROLE_003	Server & client	M	SHIP-TS-ROLE-01
TC_SHIP_SEC_001	Server or client	M	SHIP-TS-SEC-01
TC_SHIP_SEC_002	Server or client	M	SHIP-TS-SEC-02
TC_SHIP_MSG_001	Server or client	M	SHIP-TS-MSG-01 SHIP-TS-CMI-01
TC_SHIP_MSG_002	Server or client	M	SHIP-TS-MSG-02
TC_SHIP_MSG_003	Server or client	M	SHIP-IG-01 SHIP-TS-ACC-01
TC_SHIP_CMI_001	Server	M	SHIP-TS-CMI-02
TC_SHIP_CMI_002	Client	M	SHIP-TS-CMI-02
TC_SHIP_CMI_003	Server	M	SHIP-TS-CMI-04
TC_SHIP_CMI_004	Client	M	SHIP-TS-CMI-04
TC_SHIP_CMI_005	Server	M	SHIP-TS-CMI-03
TC_SHIP_CMI_006	Client	M	SHIP-TS-CMI-03
TC_SHIP_HELLO_001	Server	M	SHIP-TS-HELLO-01
TC_SHIP_HELLO_002	Server	M	SHIP-TS-HELLO-02
TC_SHIP_HELLO_003	Server	M	SHIP-TS-HELLO-03
TC_SHIP_HELLO_004	Server	M	SHIP-TS-HELLO-04
TC_SHIP_PROT_001	Server	M	SHIP-TS-PROT-01
TC_SHIP_PROT_002	Client	M	SHIP-TS-PROT-01
TC_SHIP_PROT_003	Server	M	SHIP-TS-PROT-02
TC_SHIP_PROT_004	Client	M	SHIP-TS-PROT-02
TC_SHIP_PROT_005	Server	M	SHIP-TS-PROT-03
TC_SHIP_PROT_006	Client	M	SHIP-TS-PROT-03
TC_SHIP_PIN_001	Server or client	M	SHIP-TS-PIN-01
TC_SHIP_TERM_001	Server or client	M	SHIP-TS-TERM-01
TC_SHIP_AM_001	Server or client	M	SHIP-TS-DATA-01
TC_SHIP_AMDATA_001	Server or client	M	SHIP-IG-02
TC_SHIP_AMDATA_002	Server or client	M	SHIP-IG-02
TC_SHIP_AMDATA_003	Server or client	M	SHIP-IG-03
TC_SHIP_AMDATA_004	Server or client	M	Support for TC_SHIP_AMDATA_002

512 *Table 3: Test case to requirement mapping*

513

4 Test cases

4.1 Discovery (mDNS/DNS-SD)

4.1.1 TC_SHIP_MDNS_001: Validate mDNS TXT record

Test case ID: TC_SHIP_MDNS_001

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-MDNS-01]. [SRIP-TS-TXT-01].

Description: This test case verifies that the DUT correctly announces its SHIP service via mDNS and provides a TXT record containing all mandatory key-value pairs formatted in UTF-8 according to SHIP and SRIP specifications.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort

Step	Action/Execution	Expected result
1	The test tool sends an mDNS PTR query for the SHIP service "_ship._tcp.local." and waits for the mDNS service announcement.	The DUT sends its mDNS service announcement.
2	The test tool parses the TXT record of the discovered SHIP service.	The TXT record contains the mandatory keys in valid UTF-8 formats: txtvers (exactly "1"), id (string, max 63 bytes), path (string containing "/", max 32 bytes), ski (40-digit hexadecimal string), register (boolean "true" or "false"), and cat (string of comma-separated category IDs). The optional keys brand, model and type, if present, MUST each be a string of max 32 bytes.

Table 4: TC_SHIP_MDNS_001 - Test steps

4.2 Prevent double connections

4.2.1 TC_SHIP_CONN_001: Resolve simultaneous connections by SKI (DUT has larger SKI)

Test case ID: TC_SHIP_CONN_001

Roles: DUT: Server & client. Test tool: Client & server.

Reference: [SHIP-TS-CONN-01]

Description: This test case verifies that the DUT correctly resolves simultaneous active connections to the same communication partner. The node with the larger 160-bit SKI value (the DUT) SHALL actively close the older connection and only keep the most recent connection open.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_Smaller_SKI
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TestTool_ServerAndClient

Step	Action/Execution	Expected result
1	The test tool waits passively for the DUT to actively initiate a SHIP connection (Connection A) to the test tool.	The DUT initiates a TCP connection (Connection A) to the test tool.
2	As soon as the test tool receives the incoming TCP stream for connection A, it immediately establishes a second, simultaneous SHIP connection (Connection B) to the DUT using the exact same certificate.	The DUT keeps the most recent connection (Connection B) open and continues with the SME phases on it and actively closes the older connection (Connection A) within 30s (latest before the TLS handshake of connection B completes).

Table 5: TC_SHIP_CONN_001 - Test steps

4.3 SME connection and roles

4.3.1 TC_SHIP_ROLE_001: DUT as SME server

Test case ID: TC_SHIP_ROLE_001

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-ROLE-01], [SHIP-TS-CMI-01]

Description: This test case verifies that the DUT successfully assumes the SME Server role and correctly evaluates a CMI message. Upon receiving an initial CMI message (MessageType = 0, CmiHead = 0) from the Test tool, the DUT MUST send its own CMI reply and proceed to the connection data preparation state.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established

Step	Action/Execution	Expected result
1	The test tool waits for 7 seconds after the completion of the TLS handshake.	The underlying TCP/TLS connection is still actively established.
2	The test tool sends a CMI message using PAR_cmiValidInit.	The DUT sends a CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
3	The test tool waits for an incoming SHIP message.	The DUT sends an SME "hello" message. *[1]

Table 6: TC_SHIP_ROLE_001 - Test steps

*[1] Note: The receipt of the SME "hello" message implies that the DUT successfully completed the CMI phase and entered the connection data preparation state.

4.3.2 TC_SHIP_ROLE_002: DUT as SME client

Test case ID: TC_SHIP_ROLE_002

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-ROLE-01], [SHIP-TS-CMI-01]

Description: This test case verifies that the DUT successfully assumes the SME Client role and correctly evaluates a CMI message. After actively initiating the connection and sending its initial CMI message, the DUT MUST correctly parse the test tool's CMI reply (MessageType = 0, CmiHead = 0) and proceed to the Connection data preparation state.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_TestTool_ServerOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_Ready_For_Handshake

Step	Action/Execution	Expected result
1	The test tool waits for the CMI message.	The DUT sends its CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
2	The test tool waits for 7 seconds.	The underlying TCP/TLS connection is still actively established.
3	The test tool sends a CMI message using PAR_cmiValidInit.	The DUT sends an SME "hello" message. *[1]

Table 7: TC_SHIP_ROLE_002 - Test steps

*[1] Note: The receipt of the SME "hello" message implies that the DUT successfully completed the CMI phase and entered the "Connection data preparation" state.

4.3.3 TC_SHIP_ROLE_003: Simultaneous role polymorphism

Test case ID: TC_SHIP_ROLE_003

Roles: DUT: Server & client. Test tool: Multiple instances.

Reference: [SHIP-TS-ROLE-01]

Description: This test case verifies that the DUT supports simultaneous role polymorphism by acting as a server on one connection while simultaneously acting as a client on another connection.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort

- PRE_SHIP_TestTool_Multiple_Instances
- PRE_SHIP_TestTool_ClientOnly (applied to the test tool's SHIP instance A)
- PRE_SHIP_TestTool_ServerOnly (applied to the test tool's SHIP instance B)
- PRE_SHIP_Mutual_Trust (between the DUT and the test tool's SHIP instance A)
- PRE_SHIP_Mutual_Trust (between the DUT and the test tool's SHIP instance B)
- PRE_SHIP_Ready_For_Handshake

Step	Action/Execution	Expected result
1	The test tool's SHIP instance B waits passively for the DUT to actively initiate a SHIP connection.	The DUT initiates a TCP connection to the test tool's SHIP instance B within 5 minutes after PRE_SHIP_Ready_For_Handshake is reached.
2	As soon as the test tool's SHIP instance B receives the incoming TCP connection from the DUT, the test tool's SHIP instance A immediately establishes a SHIP connection to the DUT. Both test tool instances proceed with their respective TLS and SME handshakes in parallel.	The DUT successfully establishes both connections simultaneously (acting as SME server for instance A, and SME client for instance B). The test tool receives an SME "hello" message with phase = "ready" from the DUT on both SHIP instances within 30 seconds.

Table 8: TC_SHIP_ROLE_003 - Test steps

4.4 Security

4.4.1 TC_SHIP_SEC_001: Reject spoofed certificate (no prior pairing)

Test case ID: TC_SHIP_SEC_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-SEC-01]

Description: This test case verifies that the DUT rejects a TLS connection when the remote party presents a certificate where the SKI field does not match SHA-1(public key) – a spoofed certificate. A certificate MUST satisfy: SKI = SHA-1(public key). This test covers the scenario where no prior pairing exists between DUT and test tool.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_No_Prior_Pairing
- PRE_SHIP_TestTool_SpoofedCertificate
- PRE_SHIP_Mutual_Trust*[1]

*[1] Note: This means that the DUT trusts the test tool's spoofed SKI.

Step	Action/Execution	Expected result
1	The test tool is configured as pure client. The test tool initiates a SHIP connection as client; the TLS handshake begins.	The DUT actively terminates the TLS handshake after it received the client certificate (test tool certificate).

Step	Action/Execution	Expected result
2	The test tool is configured as pure server and waits passively.	The DUT initiates a SHIP connection as client; the TLS handshake begins.*[2] The DUT actively terminates the TLS handshake after it received the server certificate (test tool certificate).

Table 9: TC_SHIP_SEC_001 - Test steps

*[2] Note: Because the DUT is configured to trust the test tool's SKI, the SHIP specification requires the DUT to cyclically attempt a connection to a discovered device with that SKI.

4.4.2 TC_SHIP_SEC_002: Reject spoofed certificate (with prior pairing)

Test case ID: TC_SHIP_SEC_002

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-SEC-02]

Description: This test case verifies that the DUT correctly detects a spoofed certificate during a reconnection attempt. After successfully pairing with a certificate, the test tool simulates an adversary and attempts to connect using a new public/private key pair but fraudulently reuses the previously trusted SKI field (SKI ≠ SHA-1(public key)). The DUT MUST detect this mismatch regardless of the SKI being known from a prior pairing.

Applicability: Applicable to any DUT.

Definitions:

- CERT_T1, CERT_T2: Certificates T1, T2.
- PUB_T1, PUB_T2: Public key fields extracted from CERT_T1, CERT_T2.
- PRV_T1, PRV_T2: Private keys (secret) of PUB_T1, PUB_T2.
- SKI_T1 = SHA-1(PUB_T1) (Note: SKI_T1 is a calculated value, i.e. not a field extracted from a certificate).

Preconditions:

- PRE_SHIP_ConnectionEstablished (The test tool uses a certificate CERT_T1 with an SKI field set to SKI_T1 = SHA-1(PUB_T1), which is trusted by the DUT.)

Step	Action/Execution	Expected result
1	The test tool closes the active SHIP connection. It then reconfigures its SHIP instance to use a spoofed certificate CERT_T2, constructed with a different key pair (PUB_T2, PRV_T2 ≠ PUB_T1, PRV_T1) but with the SKI field forged to SKI_T1. The mDNS TXT record retains the same SHIP ID and SKI_T1. The DUT is NOT reconfigured and still trusts SKI_T1. The test tool still trusts the DUT.	

Step	Action/Execution	Expected result
2	The test tool is configured as pure client. The test tool initiates a SHIP connection as client; the TLS handshake begins.	The DUT actively terminates the TLS handshake after it received the client certificate (test tool certificate).
3	The test tool is configured as pure server and waits passively.	The DUT initiates a SHIP connection as client; the TLS handshake begins.*[1] The DUT actively terminates the TLS handshake after it received the server certificate (test tool certificate).

Table 10: TC_SHIP_SEC_002 - Test steps

*[1] Note: Because the DUT still trusts SKI_T1 and the test tool advertises this SKI via mDNS, the SHIP specification requires the DUT to cyclically attempt a connection.

4.5 Basic message structure

4.5.1 TC_SHIP_MSG_001: Reject message without MessageValue

Test case ID: TC_SHIP_MSG_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-MSG-01]. [SHIP-TS-CMI-01].

Description: This test case verifies that the DUT requires a MessageValue of at least 1 octet for a SHIP message. Messages containing only a MessageType but an empty MessageValue MUST be silently discarded or actively cause a connection termination.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_ConnectionEstablished
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a SHIP message using PAR_msgEmptyMessageValue and waits for up to 3 seconds.	The DUT either actively closes the TCP/TLS connection, OR it silently ignores the message and keeps the connection open. If the DUT closes the connection, the test case is concluded successfully and step 2 is not executed.
2	The test tool sends an SME "access methods request" message.	The DUT replies with an SME "access methods" message.

Table 11: TC_SHIP_MSG_001 - Test steps

4.5.2 TC_SHIP_MSG_002: Reject unknown MessageType

Test case ID: TC_SHIP_MSG_002

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-MSG-02]

Description: This test case verifies that the DUT securely handles unknown SHIP message types. If the DUT receives a SHIP message with an undefined MessageType (e.g., 0xFF), it SHALL silently discard the message or actively close the communication channel.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_ConnectionEstablished
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a SHIP message using PAR_msgUnknownMessageType and waits for up to 3 seconds.	The DUT either actively closes the TCP/TLS connection, OR silently ignores the message and keeps the connection open. If the DUT closes the connection, the test case is concluded successfully and step 2 is not executed.
2	The test tool sends an SME "access methods request" message.	The DUT replies with an SME "access methods" message.

Table 12: TC_SHIP_MSG_002 - Test steps

4.5.3 TC_SHIP_MSG_003: Support JSON whitespace formatting

Test case ID: TC_SHIP_MSG_003

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-IG-01]. [SHIP-TS-ACC-01].

Description: This test case verifies that the DUT's JSON parser correctly interprets SHIP messages that are not "minified" and contain structural whitespace characters (Space, Horizontal tab, LF, CR) as permitted by the JSON specification.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends an SME "access methods request" message. The JSON structure is artificially formatted to contain spaces (0x20), horizontal tabs (0x09), LFs (0x0A), and CRs (0x0D) before and after structural characters (like {, }, [,], :, and ,).	The DUT replies with an SME "Access methods" message.

Table 13: TC_SHIP_MSG_003 - Test steps

4.6 SME connection mode initialization (CMI)

4.6.1 TC_SHIP_CMI_001: Reject invalid MessageType (DUT as server)

Test case ID: TC_SHIP_CMI_001

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-CMI-02]

Description: This test case verifies that the DUT follows the CMI evaluation rules. If the DUT (acting as server) receives a SHIP message with an invalid MessageType (MessageType ≠ 0) during the CMI phase, it SHALL send a CMI message with MessageValue = 0 to the client and close the connection immediately afterwards.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiInvalidMessageType.	The DUT sends a CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
2	The test tool waits for a SHIP connection termination.	The underlying TCP/TLS connection is actively closed by the DUT.

Table 14: TC_SHIP_CMI_001 - Test steps

4.6.2 TC_SHIP_CMI_002: Reject invalid MessageType (DUT as client)

Test case ID: TC_SHIP_CMI_002

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-CMI-02]

Description: This test case verifies that the DUT follows the CMI evaluation rules. If the DUT (acting as client) receives a SHIP message with an invalid MessageType (MessageType ≠ 0) in response to its initial CMI message, it SHALL close the connection immediately without sending any further messages.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_TestTool_ServerOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SHIP connection and the initial CMI message.	The DUT establishes the TCP/TLS connection and sends its initial CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
2	The test tool sends a CMI message using PAR_cmiInvalidMessageType.	The DUT does NOT send any further SHIP messages. The underlying TCP/TLS connection is actively closed by the DUT immediately.

Table 15: TC_SHIP_CMI_002 - Test steps

4.6.3 TC_SHIP_CMI_003: Apply CmiTimeout (DUT as server)

Test case ID: TC_SHIP_CMI_003

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-CMI-04]

Description: This test case verifies that the DUT (acting as server) applies the CmiTimeout (10-30s). The DUT SHALL close the connection if it does not receive the initial CMI message from the Test tool within this timeframe. The test checks both the lower bound (no disconnect before 10s) and the upper bound (disconnect before 30s).

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool waits for 7 seconds.	The underlying TCP/TLS connection is still active.
2	The test tool waits for a SHIP connection termination for up to 25 seconds (total elapsed time: 32 seconds since TLS handshake completion).	The underlying TCP/TLS connection is actively closed by the DUT within the timeframe.

Table 16: TC_SHIP_CMI_003 - Test steps

4.6.4 TC_SHIP_CMI_004: Apply CmiTimeout (DUT as client)

Test case ID: TC_SHIP_CMI_004

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-CMI-04]

Description: This test case verifies that the DUT (acting as client) applies the CmiTimeout (10-30s). The DUT SHALL close the connection if it does not receive a CMI reply from the Test tool after sending its initial CMI message. The test checks both the lower bound and the upper bound.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_TestTool_ServerOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SHIP connection and the initial CMI message.	The DUT establishes the TCP/TLS connection and sends its initial CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
2	The test tool waits for 7 seconds. The test tool does NOT send any SME message.	The underlying TCP/TLS connection is still active.
3	The test tool waits for a SHIP connection termination for up to 25 seconds (total elapsed time: 32 seconds since the end of test step 1).	The underlying TCP/TLS connection is actively closed by the DUT within the timeframe.

Table 17: TC_SHIP_CMI_004 - Test steps

4.6.5 TC_SHIP_CMI_005: Reject invalid CmiHead (DUT as server)

Test case ID: TC_SHIP_CMI_005

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-CMI-03]

Description: This test case verifies that the DUT follows the CMI evaluation rules for the CmiHead parameter. If the DUT (acting as server) receives a SHIP message with an invalid CmiHead (CmiHead > 0) during the CMI phase, it SHALL send a CMI message with MessageValue = 0 to the client and close the connection immediately afterwards.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiInvalidCmiHead.	The DUT sends a CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).

Step	Action/Execution	Expected result
2	The test tool waits for a SHIP connection termination for up to 10 seconds.	The underlying TCP/TLS connection is actively closed by the DUT.

Table 18: TC_SHIP_CMI_005 - Test steps

4.6.6 TC_SHIP_CMI_006: Reject invalid CmiHead (DUT as client)

Test case ID: TC_SHIP_CMI_006

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-CMI-03]

Description: This test case verifies that the DUT (acting as SME client) actively closes the connection if it receives a CMI message with a CmiHead greater than 0 from the server.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_TestTool_ServerOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SHIP connection and the initial CMI message.	The DUT establishes the TCP/TLS connection and sends its initial CMI message with MessageType = 0 and MessageValue = 0 (CmiHead = 0).
2	The test tool sends a CMI message using PAR_cmiInvalidCmiHead.	The DUT actively closes the TCP/TLS connection immediately.

Table 19: TC_SHIP_CMI_006 - Test steps

4.7 SME connection state "Hello"

4.7.1 TC_SHIP_HELLO_001: Process valid Hello & enter protocol handshake

Test case ID: TC_SHIP_HELLO_001

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-HELLO-01]

Description: This test case verifies that the DUT successfully processes the SME connection state "hello". After mutually exchanging SME "hello" messages with the phase "ready", the DUT MUST transition to the "protocol handshake" state and actively send an SME "protocol handshake" message.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort

- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiValidInit and waits for the DUT's initial SME "hello" message.	The DUT replies with a CMI message (MessageType = 0, CmiHead = 0) and subsequently sends its initial SME "hello" message with phase = "ready".
2	The test tool sends an SME "hello" message using PAR_helloStateReady.	The DUT sends an SME "protocol handshake" message.

Table 20: TC_SHIP_HELLO_001 - Test steps

4.7.2 TC_SHIP_HELLO_002: Accept prolongation requests

Test case ID: TC_SHIP_HELLO_002

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-HELLO-02]

Description: This test case verifies that the DUT correctly processes incoming prolongation requests. The DUT SHALL accept at least the number of prolongation requests defined by PAR_helloProlongationCount (minimum 2) and increase its Wait-For-Ready-Timer each time, preventing a premature connection abort.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiValidInit and waits for the DUT's initial SME "hello" message.	The DUT replies with a CMI message (MessageType = 0, CmiHead = 0) and subsequently sends its initial SME "hello" message with phase = "ready".
2	The test tool waits for 45 seconds.	The underlying TCP/TLS connection is still active.
3	Execute the following block of sub-steps PAR_helloProlongationCount times.	
3.1	The test tool sends an SME "hello" message using PAR_helloProlongationRequest.	The DUT sends an SME "hello" update message with phase = "ready" and maintains the connection.
3.2	The test tool waits for 60 seconds.	The underlying TCP/TLS connection is still active.

Step	Action/Execution	Expected result
4	The test tool sends an SME "hello" message using PAR_helloStateReady.	The DUT sends an SME "protocol handshake" message.

Table 21: TC_SHIP_HELLO_002 - Test steps

4.7.3 TC_SHIP_HELLO_003: Apply Wait-For-Ready-Timer (timeout)

Test case ID: TC_SHIP_HELLO_003

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-HELLO-03]

Description: This test case verifies that the DUT aborts the connection when the Wait-For-Ready-Timer expires. If the DUT does not receive a "ready" state or a prolongation request within this timeframe, it SHALL execute the common abort procedure.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiValidInit and waits for the DUT's initial SME "hello" message.	The DUT replies with a CMI message (MessageType = 0, CmiHead = 0) and subsequently sends its initial SME "hello" message with phase = "ready". The test tool verifies that the sub-element "connectionHello.waiting" is present and its value is between 58000 and 240000.
2	The test tool waits exactly the received "connectionHello.waiting" value plus 2000 milliseconds. The test tool does NOT send any SME message.	The DUT sends an SME "hello" message with phase = "aborted" (without any further sub-elements) and actively closes the TCP/TLS connection.

Table 22: TC_SHIP_HELLO_003 - Test steps

4.7.4 TC_SHIP_HELLO_004: Ignore "pending" updates without prolongationRequest in "ready" State

Test case ID: TC_SHIP_HELLO_004

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-HELLO-04]

Description: This test case verifies that the DUT follows the state machine rules for incoming pending updates. If the DUT is in the READY state and receives a "pending" message from the Test tool

without a prolongation request, it SHALL silently ignore the message and keep the connection alive until a "ready" message is received.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_TestTool_ClientOnly
- PRE_SHIP_Mutual_Trust
- PRE_SHIP_TLS_Established
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The test tool sends a CMI message using PAR_cmiValidInit and waits for the DUT's initial SME "hello" message.	The DUT replies with a CMI message (MessageType = 0, CmiHead = 0) and subsequently sends its initial SME "hello" message with phase = "ready".
2	The test tool sends an SME "hello" message using PAR_helloStatePending.	The DUT maintains the connection.
3	The test tool waits for 30 seconds and subsequently sends an SME "hello" message using PAR_helloStateReady.	The DUT sends an SME "protocol handshake" message.

Table 23: TC_SHIP_HELLO_004 - Test steps

4.8 SME protocol handshake

4.8.1 TC_SHIP_PROT_001: Support JSON-UTF8 format (DUT as server)

Test case ID: TC_SHIP_PROT_001

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-PROT-01]

Description: This test case verifies that the DUT correctly processes the 3-way protocol handshake. The DUT SHALL wait for the client's announceMax message, respond with a select message choosing the "JSON-UTF8" format, and accept the final confirmation.

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool sends an SME "protocol handshake" message using PAR_protAnnounceMax.	The DUT responds with an SME "protocol handshake" message with handshakeType = "select", containing format "JSON-UTF8" and a SHIP specification version that does not exceed the version announced by the test tool (PAR_testToolShipVersion).
2	The test tool sends the identical "select" message to confirm the selection.	The DUT sends an SME 'PIN state' message.

Table 24: TC_SHIP_PROT_001 - Test steps

4.8.2 TC_SHIP_PROT_002: Support JSON-UTF8 format (DUT as client)

Test case ID: TC_SHIP_PROT_002

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-PROT-01]

Description: This test case verifies that the DUT correctly initiates and processes the 3-way protocol handshake. The DUT SHALL actively send the announceMax message proposing "JSON-UTF8", and after receiving the Server's select message, it SHALL echo the selection back as confirmation.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SME "protocol handshake" message from the DUT.	The DUT sends an SME "protocol handshake" message with handshakeType = "announceMax" offering format = "JSON-UTF8" and the SHIP specification version defined in PAR_shipVersion.
2	The test tool responds with an SME "protocol handshake" message with handshakeType = "select", choosing the format "JSON-UTF8" and the greatest SHIP specification version supported by the test tool and the DUT (the minimum of PAR_testToolShipVersion and PAR_shipVersion).	The DUT sends the identical "select" message back to the test tool and subsequently sends an SME "PIN state" message.

Table 25: TC_SHIP_PROT_002 - Test steps

4.8.3 TC_SHIP_PROT_003: Apply Wait-Timer (DUT as server)

Test case ID: TC_SHIP_PROT_003

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-PROT-02]

Description: This test case verifies that the DUT aborts the connection when the 10-second Wait-Timer expires during the SME protocol handshake. If the DUT does not receive the initial "protocol handshake" message from the test tool within this timeframe, the DUT SHALL execute the common abort procedure with error type 1 (timeout).

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool completes the CMI and SME "hello" phases.	The DUT sends its final SME "hello" message with phase = "ready" and waits passively.
2	The test tool waits for up to 12 seconds. The test tool does NOT send any SME message.	The DUT sends an SME "protocol handshake error" message with error = 1 (timeout) and actively closes the TCP/TLS connection.

Table 26: TC_SHIP_PROT_003 - Test steps

4.8.4 TC_SHIP_PROT_004: Apply Wait-Timer (DUT as client)

Test case ID: TC_SHIP_PROT_004

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-PROT-02]

Description: This test case verifies that the DUT aborts the connection when the 10-second Wait-Timer expires during the protocol handshake. If the test tool does not reply with a select message within this timeframe, the DUT SHALL execute the common abort procedure with error type 1 (timeout).

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SME "protocol handshake" message from the DUT.	The DUT sends an SME "protocol handshake" message with handshakeType = "announceMax" offering the format "JSON-

Step	Action/Execution	Expected result
		UTF8" and the SHIP specification version defined in PAR_shipVersion.
2	The test tool completely stops sending any SHIP messages and waits for up to 12 seconds.	The DUT sends an SME "protocol handshake error" message with error = 1 (timeout) and actively closes the TCP/TLS connection.

Table 27: TC_SHIP_PROT_004 - Test steps

4.8.5 TC_SHIP_PROT_005: Reject unexpected message (DUT as server)

Test case ID: TC_SHIP_PROT_005

Roles: DUT: Server. Test tool: Client.

Reference: [SHIP-TS-PROT-03]

Description: This test case verifies that the DUT rejects messages that are out of the expected sequence during the Protocol Handshake. If the DUT receives any SME message other than the expected protocol handshake message, it SHALL execute the common abort procedure with error type 2 (unexpected message).

Applicability: Applicable to any DUT where PAR_shipSvc is set to "yes".

Preconditions:

- PRE_SHIP_ServerPort
- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool sends an SME "hello" message using PAR_helloStateReady.	The DUT sends an SME "protocol handshake error" message with error = 2 (unexpected message) and actively closes the TCP/TLS connection.

Table 28: TC_SHIP_PROT_005 - Test steps

4.8.6 TC_SHIP_PROT_006: Reject unexpected message (DUT as client)

Test case ID: TC_SHIP_PROT_006

Roles: DUT: Client. Test tool: Server.

Reference: [SHIP-TS-PROT-03]

Description: This test case verifies that the DUT rejects messages that are out of the expected sequence during the protocol handshake. If the DUT receives any SME message other than the expected "select" message, it SHALL execute the common abort procedure with error type 2 (unexpected message).

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_Hello_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Hello_Completed is fulfilled.

Step	Action/Execution	Expected result
1	The test tool waits for an incoming SME "protocol handshake" message from the DUT.	The DUT sends an SME "protocol handshake" message with handshakeType = "announceMax" offering the format "JSON-UTF8" and the SHIP specification version defined in PAR_shipVersion.
2	The test tool sends an SME "hello" message using PAR_helloStateReady.	The DUT sends an SME "protocol handshake error" message with error = 2 (unexpected message) and actively closes the TCP/TLS connection.

Table 29: TC_SHIP_PROT_006 - Test steps

4.9 PIN verification

4.9.1 TC_SHIP_PIN_001: PIN state none

Test case ID: TC_SHIP_PIN_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-PIN-01]

Description: This test case verifies that a DUT configured without a PIN correctly announces its state and transitions to the data exchange phase without requiring PIN input.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_PIN_Disabled
- PRE_SHIP_Protocol_Handshake_Completed

Entry point: The first test step is executed immediately after PRE_SHIP_Protocol_Handshake_Completed is fulfilled.

Step	Action/Execution	Expected result
1	In parallel, the test tool sends an SME "PIN state" message using PAR_pinStateNone and it waits for an incoming SME "PIN state" message from the DUT.	The DUT sends an SME "PIN state" message with pinState = "none".
2	The test tool sends an SME "access methods request" message.	The DUT replies with an SME "access methods" message.

Table 30: TC_SHIP_PIN_001 - Test steps

4.10 Connection termination

4.10.1 TC_SHIP_TERM_001: Apply maxTime during termination (DUT initiates)

Test case ID: TC_SHIP_TERM_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-TERM-01]

Description: This test case verifies that the DUT (acting as DEV-A) properly initiates a connection termination by sending a "close" message with an announce phase and a valid maxTime. It further verifies that the DUT closes the TCP/TLS connection when maxTime elapses without a "confirm".

Applicability: Applicable to any DUT where PAR_initiateClose is set to "yes".

Preconditions:

- PRE_SHIP_ConnectionEstablished
- PRE_SHIP_Manual_Message_Handling

Step	Action/Execution	Expected result
1	The tester configures the DUT to revoke the trust for the test tool.	The DUT sends an SME "close" message with phase = "announce", a valid "reason", and announces a "maxTime" value.
2	The test tool silently ignores the message (does not send a confirm message) and waits.	The DUT actively closes the TCP/TLS connection no later than the announced "maxTime" + 2s.

Table 31: TC_SHIP_TERM_001 - Test steps

4.11 Access Methods & Data Exchange

4.11.1 TC_SHIP_AM_001: Verify SHIP ID in access methods response

Test case ID: TC_SHIP_AM_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-TS-DATA-01]

Description: This test case verifies that the DUT correctly includes its unique SHIP ID in the response to an SME "access methods request" message. The DUT MUST set the accessMethods.id field to its unique SHIP ID.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends an SME "access methods request" message to the DUT.	The DUT replies with an SME "access methods" message. The field accessMethods.id MUST be present and MUST equal the DUT's SHIP ID: for a DUT with a SHIP server port (PAR_shipSvc = "yes"), the SHIP ID taken from the DUT's mDNS TXT record key "id"; otherwise the manufacturer-declared PAR_shipId.

Table 32: TC_SHIP_AM_001 - Test steps

4.11.2 TC_SHIP_AMDATA_001: Parallel processing of SME "data" while answering an "access methods" request

Test case ID: TC_SHIP_AMDATA_001

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-IG-02]

Description: This test case verifies that the DUT is capable of processing SME "data" messages concurrently with SME "access methods" requests of the test tool. The DUT MUST NOT drop incoming "data" messages while an "access methods" query of the test tool is not answered yet. To verify the concurrent message processing, the test uses SPINE messages as payload for the "data" messages.

Applicability: Applicable to any DUT.

Preconditions:

- PRE_SHIP_ConnectionEstablished

Step	Action/Execution	Expected result
1	The test tool sends the following messages in the given order and in immediate succession to the DUT, i.e. without any intentional delay and without waiting for responses between these messages: 1. An SME "data" message containing a SPINE "detailed discovery" read command*[1]. 2. An SME "access methods request" message. 3. An SME "data" message containing a SPINE "use case discovery" read command*[2].	Within 10s, the DUT sends the SME "access methods" reply AND the corresponding SPINE reply messages*[3] (without any specific order).

Table 33: TC_SHIP_AMDATA_001 - Test steps

*[1]: A SPINE "read" command of the Function "nodeManagementDetailedDiscoveryData" on the DUT's primary NodeManagement Feature (Entity 0, Feature 0).

*[2]: A SPINE "read" command of the Function "nodeManagementUseCaseData" on the DUT's primary NodeManagement Feature (Entity 0, Feature 0).

*[3]: A SPINE "reply" command of the Function "nodeManagementDetailedDiscoveryData" and a SPINE "reply" command of the Function "nodeManagementUseCaseData", both on the primary NodeManagement Feature (Entity 0, Feature 0).

4.11.3 TC_SHIP_AMDATA_002: Parallel processing of SME "data" while awaiting a delayed "access methods" response

Test case ID: TC_SHIP_AMDATA_002

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-IG-02]

Description: This test case verifies that the DUT is capable of processing SME "data" messages concurrently with (delayed) SME "access methods" responses of the test tool. The DUT MUST NOT drop incoming "data" messages while an "access methods" response of the test tool is pending. To verify the concurrent message processing, the test uses SPINE messages as payload for the "data" messages.

Applicability: Applicable to any DUT where PAR_queryAccessMethods is set to "yes".

Preconditions:

- PRE_SHIP_ConnectionEstablished
- Starting with test step 1, the test tool can process SME "data" messages initiated by the DUT with (essential) SPINE commands as payload as required by the SPINE specification any time. Such messages of the DUT are no objective of this test case and are not verified.
  Note: This does not refer to the SPINE messages explicitly mentioned in the test steps.

Step	Action/Execution	Expected result
1	The test tool waits for an SME "access methods request" message of the DUT.	Within 10s, the DUT sends the SME "access methods request" message.
2	The test tool sends the following messages in the given order and in immediate succession to the DUT, i.e. without any intentional delay and without waiting for responses between these messages: 1. An SME "data" message containing a SPINE "detailed discovery" read command*[1]. 2. An SME "access methods request" message. 3. An SME "data" message containing a SPINE "use case discovery" read command*[2]. Then, the test tool waits and sends the following message 8s after the end of test step 1: 4. An SME "access methods" message (as response to the request of test step 1).	The DUT sends the SME "access methods" reply AND – as payload of SME "data" messages – the corresponding SPINE reply messages*[3] (without any specific order): - The test tool logs a warning if this occurs later than 7s*[4] since the beginning of this test step. - The test step is considered passed if this occurs within 10s*[4] since the beginning of this test step. Otherwise, the test step is considered failed.

Table 34: TC_SHIP_AMDATA_002 - Test steps

*[1]: A SPINE "read" command of the Function "nodeManagementDetailedDiscoveryData" on the DUT's primary NodeManagement Feature (Entity 0, Feature 0).

*[2]: A SPINE "read" command of the Function "nodeManagementUseCaseData" on the DUT's primary NodeManagement Feature (Entity 0, Feature 0).

*[3]: A SPINE "reply" command of the Function "nodeManagementDetailedDiscoveryData" and a SPINE "reply" command of the Function "nodeManagementUseCaseData", both on the primary NodeManagement Feature (Entity 0, Feature 0).

*[4]: Subsequent versions of this test specification might lower this tolerance.

4.11.4 TC_SHIP_AMDATA_003: Parallel processing of SME "access methods" while awaiting a delayed "data" response

Test case ID: TC_SHIP_AMDATA_003

Roles: DUT: Server or client. Test tool: Client or server.

Reference: [SHIP-IG-03]

Description: This test case verifies that the DUT is capable of processing SME "access methods" messages concurrently with (delayed) SME "data" messages (SPINE responses) of the test tool. The DUT MUST NOT postpone the processing of incoming "access methods" messages while a "data" message (SPINE response) of the test tool is pending. To verify the concurrent message processing, the test uses SPINE messages as payload for the "data" messages.

Applicability: Applicable to any DUT where PAR_clientDetailedDiscoverySupported is set to "yes".

Preconditions:

- PRE_SHIP_ConnectionEstablished
- Starting with test step 1, the test tool can process an SME "access methods request" of the DUT as required by the SHIP specification any time. Such messages of the DUT are no objective of this test case and are not verified.

Step	Action/Execution	Expected result
1	The test tool waits for an SME "data" message containing a SPINE "detailed discovery" read command*[1] of the DUT.	Within 10s, the DUT sends the SPINE "detailed discovery" read command*[1].
2	The test tool sends an SME "access methods request" message. Then, the test tool waits for up to 8s.	The DUT sends the SME "access methods" reply.
3	The test tool sends an SME "data" message containing a SPINE "detailed discovery" reply command*[2].	The underlying TCP/TLS connection is still actively established.

Table 35: TC_SHIP_AMDATA_003 - Test steps

*[1]: A SPINE "read" command of the Function "nodeManagementDetailedDiscoveryData" on the test tool's primary NodeManagement Feature (Entity 0, Feature 0).

*[2]: A SPINE "reply" command of the Function "nodeManagementDetailedDiscoveryData" on the primary NodeManagement Feature (Entity 0, Feature 0).

4.11.5 TC_SHIP_AMDATA_004: Verify that a DUT declaring PAR_queryAccessMethods = "no" sends no "access methods request"

Test case ID: TC_SHIP_AMDATA_004

Roles: DUT: Server or client. Test tool: Client or server.

Reference: Test methodology safeguard: verifies that a DUT declaring PAR_queryAccessMethods = "no" is consistent with its declaration; supports the applicability of TC_SHIP_AMDATA_002.

Description: This test case verifies that the DUT does not send an SME "access methods request".

Applicability: Applicable to any DUT where PAR_queryAccessMethods is set to "no".

Preconditions:

- PRE_SHIP_ConnectionEstablished
- Starting with test step 1, the test tool can process SME "data" messages initiated by the DUT with (essential) SPINE commands as payload as required by the SPINE specification any time. Such messages of the DUT are no objective of this test case and are not verified.

Step	Action/Execution	Expected result
1	The test tool waits for an SME "access methods request" message of the DUT.	Within 120s, the DUT sends no SME "access methods request" message and the TCP/TLS connection is still actively established. In this case, the test case is directly considered passed. If the DUT does not send an SME "access methods request" message but closes the TCP/TLS connection during the 120s, the test case needs to be repeated. The test case is considered passed after three repetitions for this condition.

Table 36: TC_SHIP_AMDATA_004 - Test steps

